PyAnsys Units cheat sheet

Version: main

Defining a quantity

```python
from ansys.units import Quantity, UnitRegistry

meter = Quantity(value=1, units="m")

ureg = UnitRegistry()
meter = Quantity(value=1, units=ureg.m)
meter = 1 * ureg.m
```

Defining a quantity using base dimensions

```python
from ansys.units import BaseDimensions, Dimensions
from ansys.units import Quantity

dims = BaseDimensions
dimensions = Dimensions({dims.LENGTH: 1, dims.TIME: -2})

acceleration = Quantity(value=3, dimensions=dimensions)
acceleration.value  # 3.0
acceleration.units.name  # "m s^-2"
```

Defining a quantity using the quantity table

```python
from ansys.units import Quantity

torque = Quantity(5, quantity_table={"Torque": 1})

torque.value  # 5.0
torque.units.name  # "N m"
torque.units.si_units  # "kg m^2 s^-2"
```

Array quantities

```python
from ansys.units import Quantity

lengths = Quantity([1.0, 6.0, 7.0], "m")
lengths[1]  # Quantity(6.0, "m")
lengths / Quantity(2, "s")  # ([0.5 3. 3.5], "m s^-1")

grid = Quantity([[1.0, 2.0], [3.0, 4.0]], "kg")
grid.shape  # (2, 2)
grid[:, 0]  # Quantity([1. 3.], "kg")
```

Arithmetic operations

```python
from ansys.units import Quantity
import math

meter = Quantity(value=1, units="m")
foot = Quantity(value=1, units="ft")

meter * 2  # Quantity(2.0, "m")
meter / 2  # Quantity(0.5, "m")
meter**2  # Quantity(1.0, "m^2")
abs(-meter)  # Quantity(1.0, "m")

meter + foot  # Quantity(1.3048, "m")
foot + meter  # Quantity(4.28083989501312, "ft")
meter > foot  # True

math.sin(Quantity(90, "degree"))  # 1.0
```

Unit arithmetic and comparisons

```python
from ansys.units import Quantity, UnitRegistry

ureg = UnitRegistry()
ureg.kg * ureg.m  # Unit("kg m")
ureg.m / ureg.s  # Unit("m s^-1")
5 * ureg.N  # Quantity(5.0, "N")

Quantity(1, "m") == Quantity(100, "cm")  # True
Quantity(1, "m") < Quantity(1, "s")  # raises error
```

Pre-defined unit systems

```python
from ansys.units import UnitSystem

si = UnitSystem(system="SI")
si.LENGTH # "m"

cgs = UnitSystem(system="CGS")
cgs.LENGTH # "cm"

bt = UnitSystem(system="BT")
bt.LENGTH # "ft"
```

Custom unit systems

```python
from ansys.units import BaseDimensions, UnitSystem
from ansys.units import UnitRegistry

ureg = UnitRegistry()
dims = BaseDimensions

sys = UnitSystem(
    base_units={
        dims.MASS: ureg.slug,
        dims.LENGTH: ureg.ft,
        dims.TEMPERATURE: ureg.R,
        dims.TEMPERATURE_DIFFERENCE: ureg.delta_R,
        dims.CHEMICAL_AMOUNT: ureg.slugmol,
    }
)

sys.LENGTH  # Unit("ft")
```

Convert quantity to other units

```python
from ansys.units import Quantity

flbs = Quantity(1, "lb ft^-1 s^-1")
flbs.value  # 1

pas = flbs.to("Pa s")
pas.value  # 1.4881639435695542
pas.units.name  # 'Pa s'
```

Convert units using unit systems

```python
from ansys.units import Unit, UnitSystem

si = Unit("m s^-1")
bt = UnitSystem(system="BT")
si.convert(bt)  # Unit("ft s^-1")
```

Preferred units conversion policy

```python
from ansys.units import BaseDimensions, PreferredUnits
from ansys.units import Quantity, UnitSystem

dims = BaseDimensions
mm_t_s = UnitSystem(
    base_units={
        dims.MASS: "tonne",
        dims.LENGTH: "mm",
        dims.TIME: "s",
    }
)
stress = Quantity(210e9, "Pa")

stress.convert(mm_t_s).units.name
# "tonne mm^-1 s^-2"

prefs = PreferredUnits(["MPa"])  # matched by dimension
stress.convert(mm_t_s, preferred_units=prefs)
# Quantity(210000.0, "MPa")
```

Named unit systems

```python
from ansys.units import BaseDimensions, Quantity
from ansys.units import UnitSystem

dims = BaseDimensions
UnitSystem.register_system(
    name="MMGS",
    base_units={
        dims.MASS: "g",
        dims.LENGTH: "mm",
        dims.TIME: "s",
        dims.TEMPERATURE: "K",
        dims.TEMPERATURE_DIFFERENCE: "delta_K",
        dims.ANGLE: "radian",
        dims.CHEMICAL_AMOUNT: "mol",
        dims.LIGHT: "cd",
        dims.CURRENT: "A",
        dims.SOLID_ANGLE: "sr",
    },
)
mmgs = UnitSystem(system="MMGS")
Quantity(1, "N").convert(mmgs)
# Quantity(1000000.0, "g mm s^-2")
```

Preferred units when creating quantities

```python
from ansys.units import Quantity

Quantity.preferred_units(units=["J"])
Quantity(1, "N m")  # Quantity(1.0, "J")
Quantity.preferred_units(units=["J"], remove=True)
```

Temperature and temperature difference

```python
from ansys.units import Quantity

K = Quantity(280, "K")
K2 = Quantity(295, "K")
delta_K = Quantity(5, "delta_K")

K2 - K  # Quantity(15.0, "delta_K")
K + delta_K  # Quantity(285.0, "K")
K2 - delta_K  # Quantity(290.0, "K")
delta_K + delta_K  # Quantity(10.0, "delta_K")
K + K2  # raises ProhibitedTemperatureOperation
```

Temperature values below absolute zero

```python
from ansys.units import Quantity, Unit

Quantity(-2, "K")  # Quantity(-2.0, "delta_K")
Quantity(-300, "C")  # Quantity(-300.0, "delta_C")
Quantity(-2, "C")  # Quantity(-2.0, "C")
Unit("F").min_absolute_value  # -459.67
```

Offset units (C and F)

```python
from ansys.units import Quantity

t = Quantity(20, "C")
t * 2  # raises ProhibitedOffsetUnitOperation
t.to("K") * 2  # Quantity(586.3, "K")
Quantity(5, "delta_C") * 2  # Quantity(10.0, "delta_C")

# In compound units, C is a temperature difference
k = Quantity(2.0, "W m^-1 C^-1")
k.to("W m^-1 K^-1")  # Quantity(2.0, "W m^-1 K^-1")
```

Angle as non-dimension

```python
import ansys.units as pyunits

degree = pyunits.Quantity(1.0, "degree")
degree.is_dimensionless  # True
degree + 1  # Quantity(58.29577951308232, "degree")
```

Angle as dimension

```
import os

os.environ["PYANSYS_UNITS_ANGLE_AS_DIMENSION"] = "1"
import ansys.units as pyunits

degree = pyunits.Quantity(1.0, "degree")
degree.is_dimensionless  # False
degree + 1  # Not allowed
```

Unit string format

```
from ansys.units import Unit

# <multiplier prefix>[unit]<^exponent>
accel = Unit("m s^-2")  # base units with exponent
density = Unit("kg m^-3")  # compound unit string
kpa = Unit("kPa")  # prefix "k" + "Pa"
```

Unit properties

```
from ansys.units import Quantity, Unit, get_si_value

psi = Unit("psi")
psi.si_units  # "kg m^-1 s^-2"
psi.si_scaling_factor  # 6894.757293168363
psi.dimensions  # {"MASS": 1.0, "LENGTH": -1.0, ...}
psi.compatible_units()  # {"Pa", "bar", "psf", ...}
get_si_value(Quantity(1, "psi"))  # 6894.757293168363
```

Unit aliases

```
from ansys.units import Quantity, Unit, UnitRegistry

# Built-in aliases resolve to canonical names
Unit("deg")  # "degree"
Unit("meter")  # "m"
Unit("deg sec^-1")  # "degree s^-1"
Quantity(180, "deg").to("rad")  # ~3.14159 radian

# Register a custom alias
ureg = UnitRegistry()
ureg.register_alias("angular_deg", "degree")
Unit("angular_deg")  # resolves to "degree"
```

Registering custom units

```
from ansys.units import UnitRegistry, get_si_value
from ansys.units.common import m

ur = UnitRegistry(
    custom_units=[
        {
            "unit": "micron",
            "composition": "m",
            "factor": 1e-6,
        },
    ]
)
q = ur.Quantity(1, "micron")  # string-based lookup
get_si_value(q)  # 1e-06

ur.register_unit(unit="Q", composition="N m", factor=1)
ur.Q == ur.J  # True

# Object-based form, with an alias
ur.register_unit("furlong", 201.168 * m,
aliases=["furlongs"])
ur.furlong is ur.furlongs  # True

# Re-registering the same name with an equivalent
definition is a no-op
ur.register_unit(unit="Q", composition="W s", factor=1)
# Returns existing ur.Q
```

Persisting custom units

```
from ansys.units import UnitRegistry, load_custom_units
from ansys.units.common import m

ur = UnitRegistry()
ur.register_unit("furlong", 201.168 * m,
aliases=["furlongs"])
ur.export_custom_units("my_units.yaml")  # Not written to
cfg.yaml

ur2 = UnitRegistry(custom_units_file="my_units.yaml")
ur2.furlong is ur2.furlongs  # True
```

Quantity dimensions

```
from ansys.units import QuantityDimensions

qd = QuantityDimensions
qd.PRESSURE == qd.FORCE / qd.AREA  # True
```

Variable descriptors

```
from ansys.units import MappingConversionStrategy
from ansys.units import VariableCatalog

p = VariableCatalog.PRESSURE
p.name  # "pressure"
p.dimension  # {"MASS": 1.0, "LENGTH": -1.0, ...}


class MyNames(MappingConversionStrategy):
    _mapping = {VariableCatalog.PRESSURE: "p"}


MyNames().to_string(p)  # "p"
MyNames().to_variable_descriptor("p") == p  # True
```

Common errors

```
from ansys.units import Quantity, Unit

Quantity(1, "m").to("s")  # IncompatibleDimensions
Quantity(1, "m") + Quantity(1, "s")  # IncorrectUnits
Unit("not_a_unit")  # UnconfiguredUnit
Quantity(1, "K") + Quantity(1, "K")
# ProhibitedTemperatureOperation
Quantity(1, "C") * 2  # ProhibitedOffsetUnitOperation
Unit("kC")  # PrefixedOffsetUnit
```

NumPy functions

```
import numpy as np
from ansys.units import Quantity

x = Quantity(np.array([1.0, 2.0, 3.0, 6.0]), "m")
np.mean(x)  # Quantity(3.0, "m")
np.var(x)  # Quantity(3.5, "m^2")
np.sqrt(Quantity(9.0, "m^2"))  # Quantity(3.0, "m")
np.maximum(x, Quantity(250, "cm"))  # in "m"
np.concatenate([x, Quantity([300.0], "cm")])  # in "m"

np.sin(Quantity(90.0, "degree"))  # Quantity(1.0, "")
np.arccos(Quantity(0.0, ""))  # (1.5707..., "radian")
np.exp(Quantity(1.0, "m"))  # raises, not dimensionless

t = Quantity(np.array([10.0, 20.0, 40.0]), "C")
np.ptp(t)  # Quantity(30.0, "delta_C")
np.sum(t)  # raises ProhibitedTemperatureOperation
```

Matrix multiplication

```
import numpy as np
from ansys.units import Quantity

A = Quantity(np.array([[1.0, 2.0], [3.0, 4.0]]), "m")
v = Quantity(np.array([1.0, 1.0]), "N")
A @ v  # Quantity([3. 7.], "m N")
A @ [1.0, 2.0]  # Quantity([5. 11.], "m")
```

Serialization

```
import json
from ansys.units import Quantity

q = Quantity([1.0, 2.0], "m s^-1", source="sensor")
data = q.to_dict()
# {"value": [1.0, 2.0], "units": "m s^-1",
#  "source": "sensor"}
Quantity.from_dict(data)  # same quantity
Quantity.from_dict(json.loads(json.dumps(data)))

Quantity.from_string("5.0 m s^-1")  # parses str(q)
```

Pydantic models

```
from pydantic import BaseModel
from ansys.units import Quantity


class Measurement(BaseModel):
    length: Quantity


m = Measurement(length=Quantity(2.0, "m"))
text = m.model_dump_json()
Measurement.model_validate_json(text)
```

Custom units when loading and unpickling

```
import pickle
from ansys.units import UnitRegistry
from ansys.units import set_application_registry

ur = UnitRegistry()
ur.register_unit(
    unit="micron", composition="m", factor=1e-6
)
set_application_registry(ur)  # used to resolve names

q = ur.Quantity(3.0, "micron")
pickle.loads(pickle.dumps(q))  # Quantity(3.0, "micron")
```

Plotting with Matplotlib

```
import matplotlib.pyplot as plt
from ansys.units import Quantity

x = Quantity([1, 2, 3], "m")
y = Quantity([4, 5, 6], "kg")
fig, ax = plt.subplots()
ax.plot(x, y)
plt.show()
```

Pandas integration

```
import pandas as pd
import ansys.units  # registers the "quantity" dtype

m = pd.Series([1.0, 2.0, 3.0], dtype="quantity[m]")
m.units.to("cm")  # dtype "quantity[cm]"

df = pd.DataFrame(
    {
        "length": pd.Series([1.0], dtype="quantity[m]"),
        "mass": pd.Series([2.0], dtype="quantity[kg]"),
    }
)
```